

Take-Home Exercise: Multi-Supplier Order Router

Overview

Build a service that routes orders to suppliers based on business rules. You'll work with real-world messy data and make intelligent routing decisions that balance multiple concerns. Do not take more than 4 hours to do this.

The Problem

When we receive a multi-item order, we need to determine which supplier(s) should fulfill it. Some suppliers can handle entire orders, others specialize in specific product categories. Your job is to build the routing logic.

What We're Providing

You'll find three files in this directory:

1. **suppliers.csv** - List of suppliers with their capabilities (see details below)
2. **products.csv** - List of 200 products mapping product codes to categories
3. **sample_orders.json** - 3 example orders to test your routing logic

Supplier Data Structure

The suppliers.csv contains realistic, somewhat messy data:

- **supplier_id**: Unique identifier (e.g., "SUP-001")
- **supplier_name**: Business name
- **service_zips**: ZIP codes served - can be:
 - Explicit list: "10001, 10002, 10003"
 - Range: "10001-10100"
- **product_categories**: Comma-separated list of categories they handle
- **customer_satisfaction_score**: Rating 1-10, or "no ratings yet"

- **can_mail_order?:** "y" or "n" (whether they ship nationally)

Note: When an order's `mail\_order` attribute is false, only route to suppliers serving the customer's ZIP code. When `mail\_order` is true, any supplier with can_mail_order? = "y" is eligible regardless of ZIP code.

Requirements

Core Functionality

Build a service that:

1. Accepts an order with multiple line items
2. Reads supplier and product data from the provided CSV files
3. Implements routing logic that considers:
 - a. **Product capabilities:** Can the supplier fulfill this product category?
 - b. **Geographic coverage:** Does the supplier serve this ZIP code? (unless mail_order is true)
 - c. **Mail order eligibility:** If order allows mail order, consider suppliers with can_mail_order=y
 - d. **Customer experience:** Minimize number of shipments when possible
 - e. **Quality considerations:** Factor in customer satisfaction scores
4. **Returns a routing decision** showing which items go to which suppliers

Routing Logic Priorities

Your router should optimize for (in order of importance):

1. **Feasibility:** Only route to suppliers who can actually fulfill the items
2. **Customer experience:** Prefer fewer shipments (consolidate with one supplier when possible)
3. **Quality:** When multiple options exist, prefer higher-rated suppliers
4. **Geographic preference:** Prefer local suppliers over mail-order when ratings are similar

Endpoint Format

The order routing endpoint should be at:

POST /api/route

Example request payload:

```
{
  "order_id": "ORD-EXAMPLE",
  "customer_zip": "10015",
  "mail_order": false,
  "items": [
    { "product_code": "WC-STD-001", "quantity": 1 },
    { "product_code": "OX-PORT-024", "quantity": 1 }
  ]
}
```

POST /api/route always returns HTTP 200. The feasible field tells you whether routing succeeded.

Successful Routing Example

```
{
  "feasible": true,
  "routing": [
    {
      "supplier_id": "SUP-005",
      "supplier_name": "Respiratory Care Co Co",
      "items": [
        {
          "product_code": "WC-STD-001",
          "quantity": 1,
          "category": "wheelchair",
          "fulfillment_mode": "local"
        }
      ]
    }
  ]
}
```

Unsuccessful Routing Example

```
{
  "feasible": false,
  "errors": [
    "Order must include at least one line item.",
    "Order must include a valid customer_zip."
  ]
}
```

Deliverables

Submit a ZIP file or git repo containing:

1. Your code, containerized so we can run it
2. README.md explaining how to build and test your service
3. AI_PROMPTS.md containing your significant prompts in the order you used them